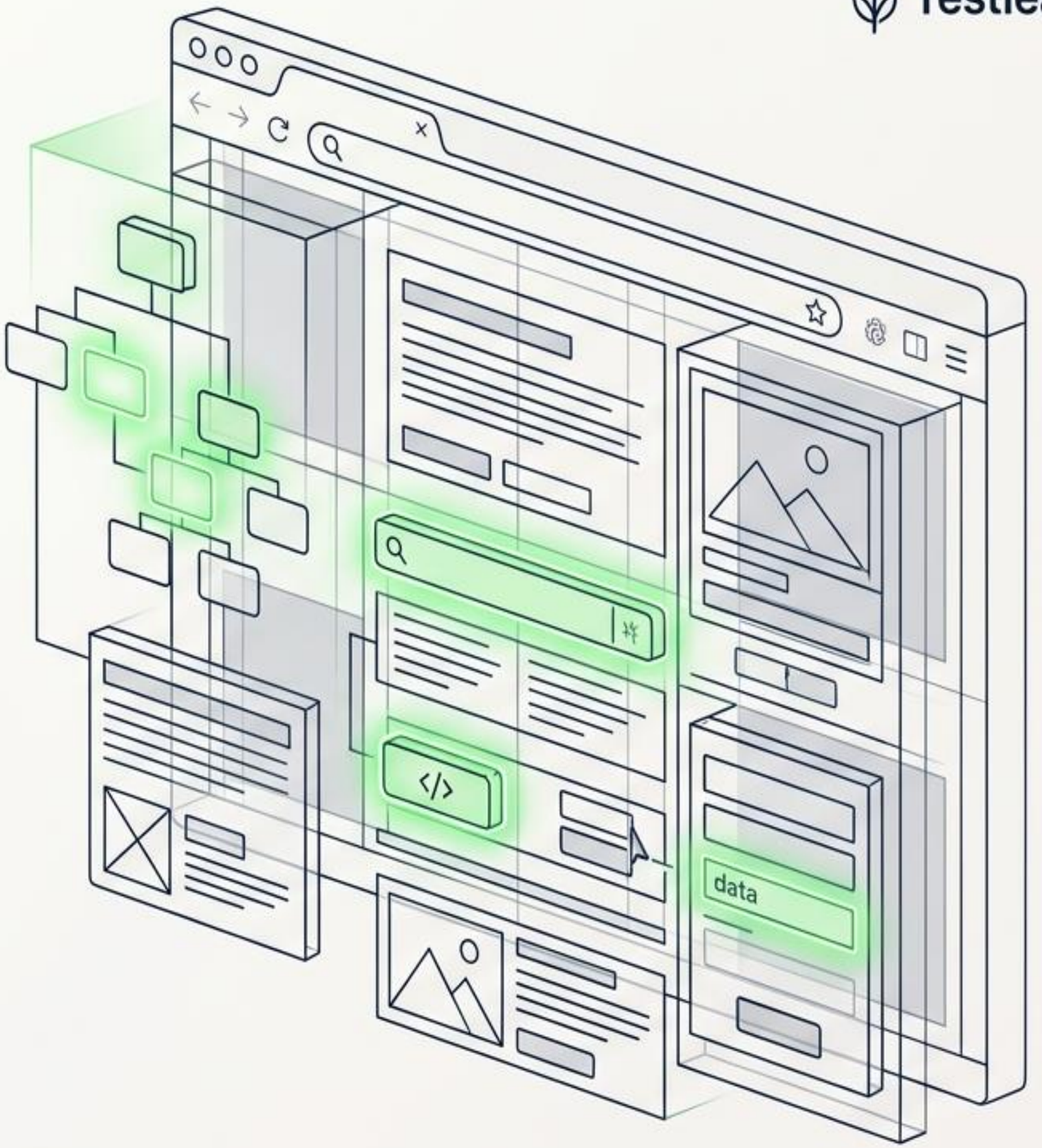
Testleaf

Mastering Masterin Locators & Selectors in Playwright

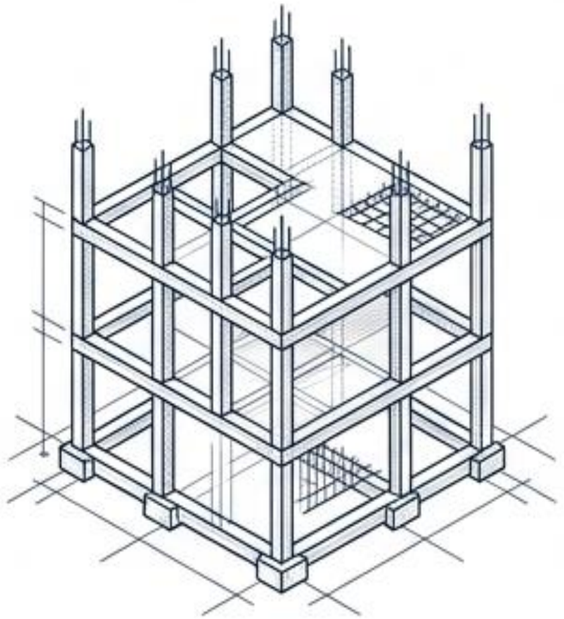
The definitive guide to navigating the DOM and targeting web elements with precision.



Testleaf

The Web Trinity: Building the House

HTML (HyperText Markup Language)



The Blueprint.

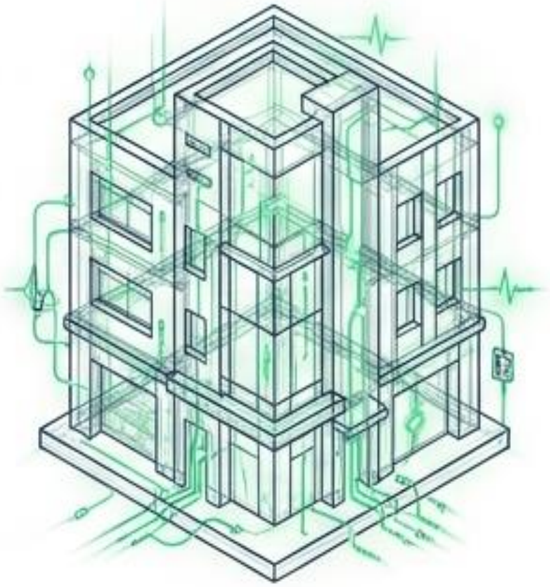
Defines structure and content (Headers, paragraphs, images, forms).

CSS (Cascading Style Sheets)

The Paint.

Controls visual styling (Fonts, colors, layouts, animations).

JavaScript

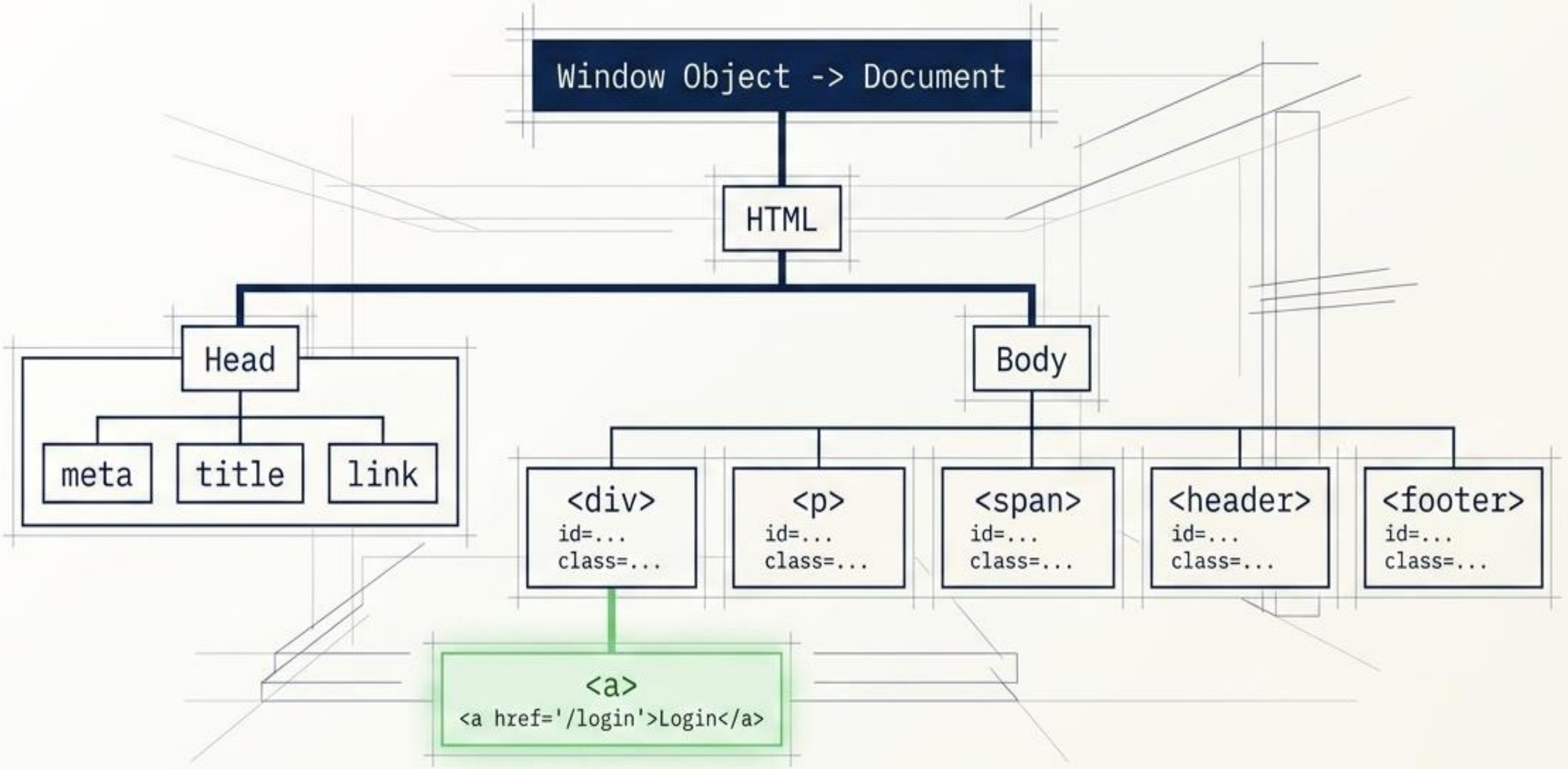


The Electricity.

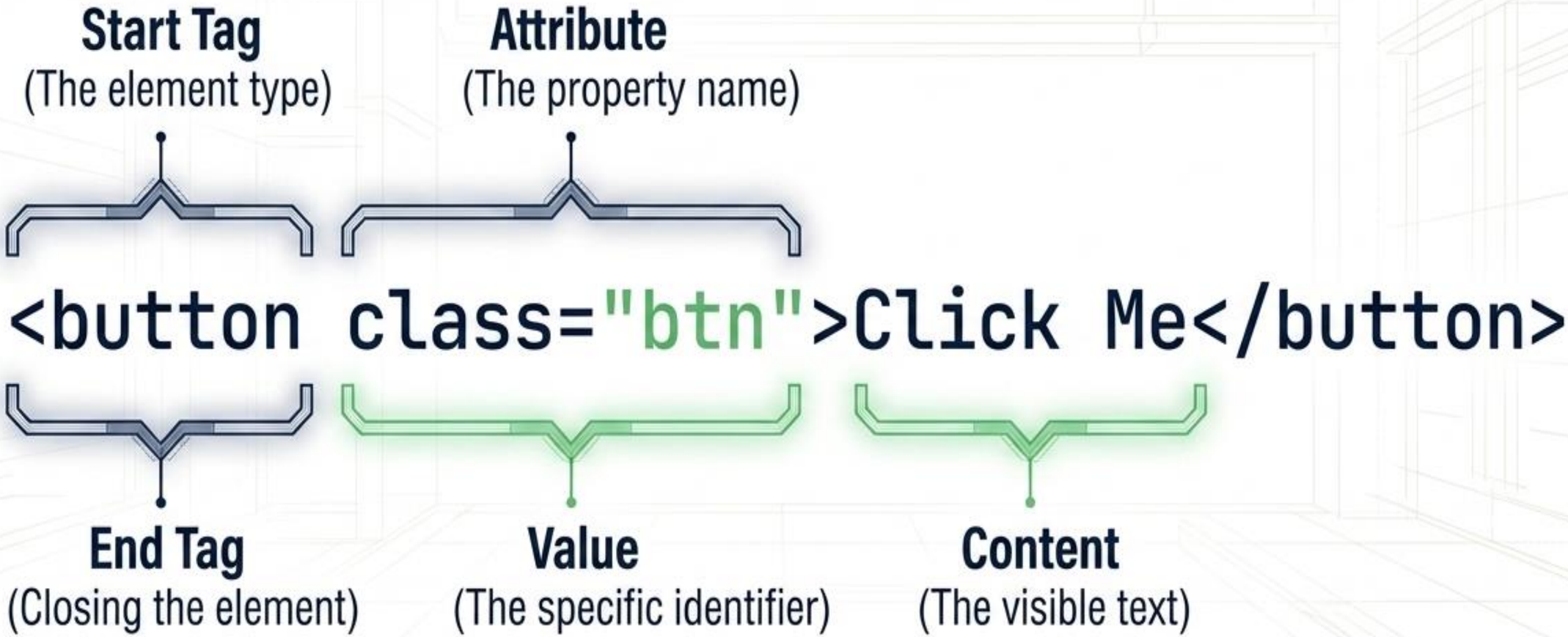
Makes it dynamic (Form validation, real-time updates).

The DOM Tree: Reading the Blueprint

The Document Object Model (DOM) transforms HTML into a structured hierarchy of nodes, allowing tools like Playwright to navigate the page programmatically.



X-Raying an HTML Element



Testleaf

The Mechanism vs. The Query

```
page.locator(...)
```

Locator

(The GPS). Playwright's method to resolve an element on the page. It manages the action and handles dynamic changes.


```
'#user-name'
```

Selector

(The Address). The specific string (CSS or XPath) passed into the locator to find the exact DOM element.

The Foundation: Basic CSS Selectors

Tag Name

Targets element type. Syntax: tagName

```
page.locator('input')
```

selects <input> elements.

ID

Targets unique identifiers. Syntax: #idValue

```
page.locator('#username')
```

selects id="username".

Class

Targets shared styling classes. Syntax: .classValue

```
page.locator('.inputLogin')
```

selects class="inputLogin".

Attribute

Targets specific properties. Syntax: [attribute='value']

```
page.locator("[name='USERNAME']")
```

selects name="USERNAME".

Vertical Navigation: Descendants & Children

Testleaf

Descendant Combinator (Space)

Selects all elements nested inside an ancestor, regardless of depth. Syntax: `form input`

Child Combinator (>)

Selects only the immediate, direct children of a parent. Syntax: `p > input`

Horizontal Navigation: Sibling Combinators

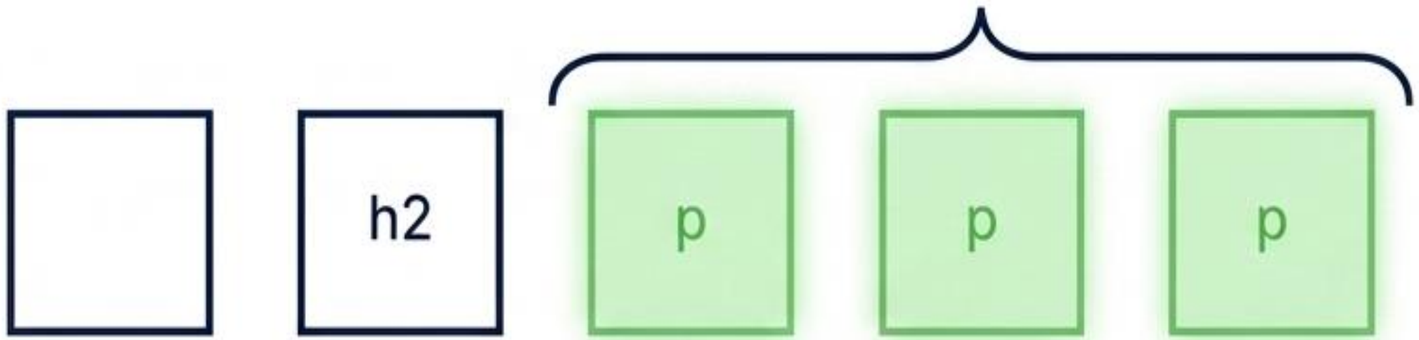
Testleaf

Adjacent Sibling (+)

Selects the immediate next sibling element. Example: label + input finds the input directly following a label.

General Sibling (~)

Selects all subsequent siblings of the specified element. Example: h2 ~ p finds all paragraphs following an h2 on the same level.



Substring Matching: Handling Dynamic Attributes

Testleaf

Starts With (^) Selects attributes beginning with a value. Syntax: [class^='inventory']

Contains (*)

Selects attributes containing a substring anywhere. Syntax: [class*='btn']

Ends With ($)

Selects attributes ending with a value. Syntax: [class$='inventory']

Execution Strategies: Page Fixture vs. CLI

Feature	Using page Fixture	Manual Setup
Browser Creation	Managed automatically by Playwright	Requires manual launch code
Context	Fresh, isolated context per test	Context persists across tests
CLI Options	Accepts CLI overrides & playwright.config.ts	Must be coded manually
Best For	Simple tests, parallel execution	Full browser control, persistent sessions

The Control Room: playwright.config.ts

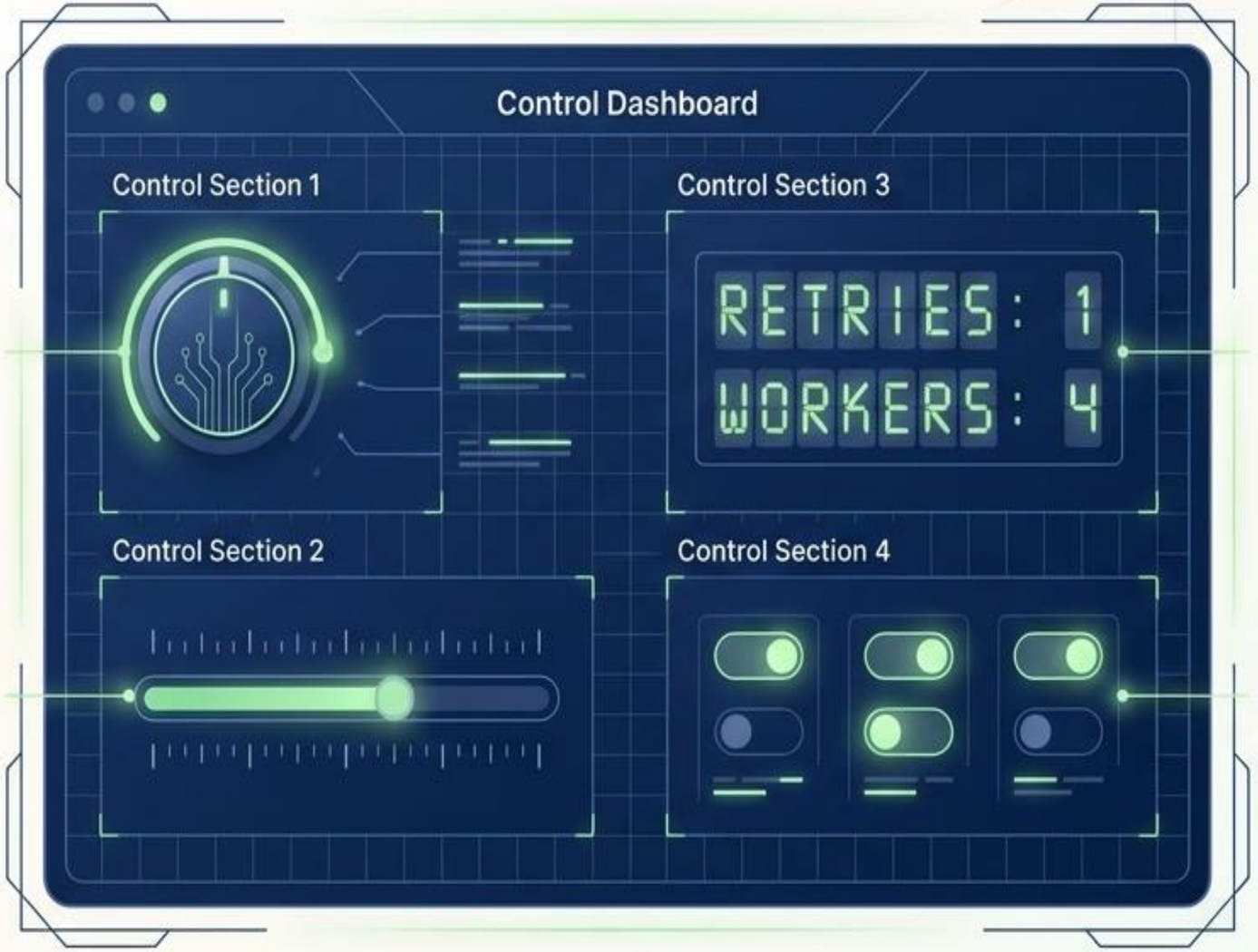
Testleaf

fullyParallel

Toggles parallel execution of files/tests.

Timeouts

Sets max time for test execution and expect assertions.



Retries & Workers

Defines retry attempts on failure and worker split.

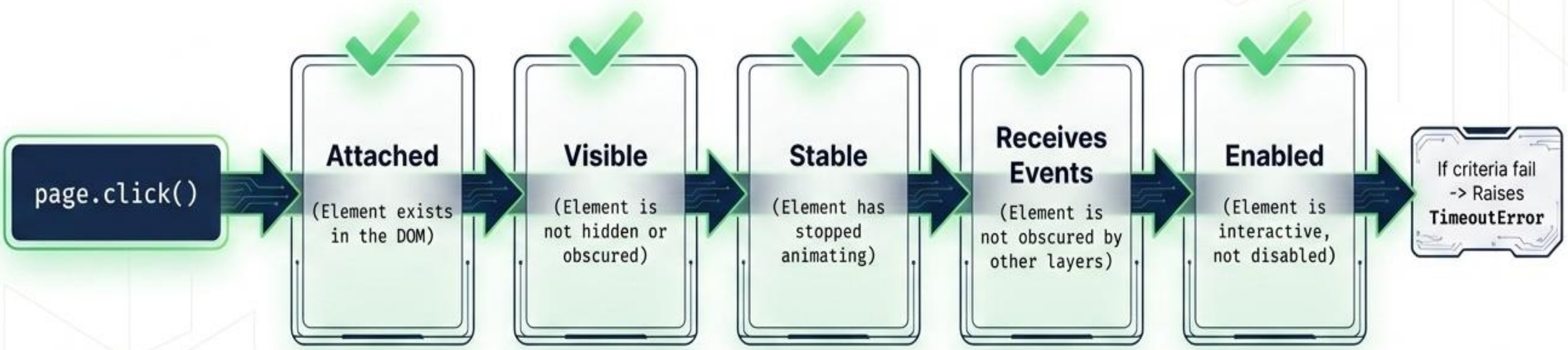
Artifacts

Conditions for saving traces, screenshots, and videos.

The Auto-Wait Lifecycle: Before the Click

Testleaf

Playwright automatically ensures all checks pass within a set timeout before performing an action.



Testleaf

Blueprint to Execution

Phase 1: Map the Structure

Understand the HTML/CSS/JS architecture and locate the exact element in the DOM tree.

Phase 2: Target Precisely

Craft resilient CSS selectors (ID, Class, Combinators, Substrings) to query the element reliably.

Phase 3: Execute with Confidence

Leverage Playwright's fixtures, global configs, and auto-wait lifecycles to ensure stable automation.